Screenshots Required:

Phase	ID	What to Capture
1	1A	`./test_objects` output showing all tests passing
1	1B	`find .pes/objects -type f` showing sharded directory structure
2	2A	`./test_tree` output showing all tests passing
2	2B	`xxd` of a raw tree object (first 20 lines)
3	3A	`pes init` → `pes add` → `pes status` sequence
3	3B	`cat .pes/index` showing the text-format index
4	4A	`pes log` output with three commits
4	4B	`find .pes -type f | sort` showing object growth
4	4C	`cat .pes/refs/heads/main` and `cat .pes/HEAD`
Final	--	Full integration test (`make test-integration`)

1A:

```
root@c1ad90d47c13:/workspace# make test_objects
./test_objects
gcc -Wall -Wextra -O2 -I/opt/homebrew/include -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -I/opt/homebrew/include -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto -L/opt/homebrew/lib
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
root@c1ad90d47c13:/workspace#
```

1B:

```
root@c1ad90d47c13:/workspace# find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
root@c1ad90d47c13:/workspace#
```

2A:

```
root@c1ad90d47c13:/workspace# make test_tree
./test_tree
gcc -Wall -Wextra -O2 -I/opt/homebrew/include -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto -L/opt/homebrew/lib
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
root@c1ad90d47c13:/workspace#
```

2B:

```
root@c1ad90d47c13:/workspace# make test_objects
./test_objects
xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
root@c1ad90d47c13:/workspace#
```

3A:

```
root@c1ad90d47c13:/workspace# echo "world" > file2.txt
./pes add file2.txt
./pes status
cat .pes/index
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  object.c
  untracked:  pes.c
  untracked:  .DS_Store
  untracked:  Makefile
  untracked:  test_objects
  untracked:  tree.h
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  test_tree
  untracked:  README.md
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  commit.h
  untracked:  tree.c

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776671066 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776671112 6 file2.txt
root@c1ad90d47c13:/workspace#
```

3B:

```
root@c1ad90d47c13:/workspace# cat .pes/index
100644 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 1776671440 12 hello.txt
100644 e67ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc 1776671440 8 bye.txt
root@c1ad90d47c13:/workspace#
```

4A:

```
root@c1ad90d47c13:/workspace# rm -rf .pes
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 9156d008d95c... Initial commit
Committed: efc2a1a8ec54... Add world
Committed: 6fd6628063af... Add farewell
commit 6fd6628063afc9cee40c74952b0bc0f08d42532ebebb9a8558dc3918a812338e
Author: Your Name <PES1UG24CS410>
Date:   1776671440

    Add farewell

commit efc2a1a8ec54b5aec74051da24d9b4b6685d1dd66bfc85b36061828b2f6ab60e
Author: Your Name <PES1UG24CS410>
Date:   1776671440

    Add worldell

commit 9156d008d95cac37c040ae37b0c83af026da5fc455ddc60cf7ffae651e943b04
Author: Your Name <PES1UG24CS410>
Date:   1776671440

    Initial committial comQ

root@c1ad90d47c13:/workspace#
```

4B:

```
root@c1ad90d47c13:/workspace# find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6f/d6628063afc9cee40c74952b0bc0f08d42532ebebb9a8558dc3918a812338e
.pes/objects/91/56d008d95cac37c040ae37b0c83af026da5fc455ddc60cf7ffae651e943b04
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ef/c2a1a8ec54b5aec74051da24d9b4b6685d1dd66bfc85b36061828b2f6ab60e
.pes/refs/heads/main
root@c1ad90d47c13:/workspace#
```

4C:

```
root@c1ad90d47c13:/workspace# cat .pes/refs/heads/main
cat .pes/HEAD
6fd6628063afc9cee40c74952b0bc0f08d42532ebebb9a8558dc3918a812338e
ref: refs/heads/main
root@c1ad90d47c13:/workspace#
```

Final:

```
root@c1ad90d47c13:/workspace# make test-integration
gcc -Wall -Wextra -O2 -I/opt/homebrew/include -c tree.c -o tree.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto -L/opt/homebrew/lib
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 740504bca670... Initial commit

Log after first commit:
commit 740504bca670a58bbe4970f29dcf1c49ab0b8c1a72c4170e83e44fd4f2d8236e
Author: Your Name <PES1UG24CS410>
Date:   1776671516

    Initial commit


--- Second Commit ---
Committed: dd856bfa2f85... Update file.txt

--- Third Commit ---
Committed: 47b8cf720ad7... Add farewell
```

```
Committed: 47b8cf720ad7... Add farewell

--- Full History ---
commit 47b8cf720ad735cdab4c02795fec9442540750218bbba66dcdc279a808ba724f
Author: Your Name <PES1UG24CS410>
Date:   1776671516

    Add farewell

commit dd856bfa2f855e8177c59ee6a37bfbe3d26390b01db4ead359f7158fc02d743e
Author: Your Name <PES1UG24CS410>
Date:   1776671516

    Update file.txt

commit 740504bca670a58bbe4970f29dcf1c49ab0b8c1a72c4170e83e44fd4f2d8236e
Author: Your Name <PES1UG24CS410>
Date:   1776671516

    Initial committial comQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
47b8cf720ad735cdab4c02795fec9442540750218bbba66dcdc279a808ba724f

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/47/b8cf720ad735cdab4c02795fec9442540750218bbba66dcdc279a808ba724f
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/74/0504bca670a58bbe4970f29dcf1c49ab0b8c1a72c4170e83e44fd4f2d8236e
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/dd/856bfa2f855e8177c59ee6a37bfbe3d26390b01db4ead359f7158fc02d743e

=== All integration tests completed ===
```

Questions:

Q5.1 — How would you implement `pes checkout <branch>`?

To switch branches, three things need to change in `.pes/`:

1. HEAD gets updated to point to the new branch: `ref: refs/heads/<branch>`
2. The new branch file (`.pes/refs/heads/<branch>`) is read to get the target commit hash
3. The working directory files are updated to match the target commit's tree

The complex part is step 3 — you have to walk the target commit's tree, compare each file to what's currently on disk, and overwrite files that differ. You also need to delete files that exist in the current branch's tree but not in the target, and create files that exist in the target but not currently. This requires reading both the current HEAD tree and the target tree from the object store and diffing them.

Q5.2 — How would you detect a "dirty working directory" conflict during checkout?

Before switching, for every file tracked in the current index, recompute its blob hash from the working directory and compare it to the hash stored in the index. If they differ, the file has been modified but not staged — this is a conflict if the target branch has a different version of that file. You can detect this

purely using the index (for staged vs working dir comparison) and the object store (for comparing the current branch's committed version vs the target branch's version of the same path).

Q5.3 — What is "detached HEAD" and how do you recover commits made in that state?

Detached HEAD means `.pes/HEAD` contains a raw commit hash instead of `ref: refs/heads/<branch>`. If you make commits in this state, each new commit is written to the object store and HEAD is updated to the new hash — but no branch pointer moves. So when you switch away, those commits become unreachable from any branch. To recover them, you need the commit hash (visible in terminal history or `pes log` output from that session), then create a new branch pointing to it: write the hash into `.pes/refs/heads/recovery` and update HEAD to point to that branch.

Q6.1 — Algorithm to find and delete unreachable objects (garbage collection)

Start from all branch files in `.pes/refs/heads/` — these are your GC roots. For each branch, read the commit it points to, then walk the commit chain following parent pointers until you hit the first commit. For each commit visited, add its hash to a reachable set, then add its tree hash, and recursively add all blob and subtree hashes in that tree. Once you've visited all branches, scan every file in `.pes/objects/` and delete any whose hash is not in the reachable set. The right data structure for the reachable set is a hash set (like a C hashtable or a sorted array with binary search) for O(1) or O(log n) lookup. For 100,000 commits across 50 branches, you'd visit roughly 100,000 commits + ~100,000 trees + several hundred thousand blobs — so on the order of a few hundred thousand to a million objects total.

Q6.2 — Race condition between GC and concurrent commit

The race: GC scans the object store and marks a blob as unreachable because no commit references it yet. Simultaneously, a commit operation has written that blob to the store but hasn't yet updated the index or created the commit object. GC deletes the blob. The commit then tries to build a tree referencing that blob — it's gone, and the commit is now broken. Git avoids this by using a "grace period" — objects newer than a threshold (default 2 weeks) are never deleted by GC regardless of reachability. This means a concurrent commit's freshly written objects are always safe. Additionally, Git uses lockfiles on refs so that GC can check whether any ref update is in progress before deleting anything.